Test Plan

One section per hard problem, in the same order as the design document. For each: what the suite asserts, and — the part that matters — **how each test fails if the design is wrong** in the way I am most worried about.

All three problems have real pytest files in tests/, running against a **live in-process Iceberg catalog**: Spark local[2], hadoop catalog type, a tmp warehouse, iceberg-spark-runtime pinned in conftest.py. The assertions are against real MERGE, real branch DDL and real fast_forward semantics, not mocks of them, which is the only way a test can catch a wrong API rather than a wrong idea.

> **The suite is written against a real catalog but has not been executed — see the final section. I would rather state that than have it discovered.**

1. CDC replay safety — tests/test_cdc_replay_idempotency.py

The suite deliberately separates two mechanisms, because they carry different weight: **the LSN predicate makes a replay correct; the marker table only makes it cheap.** So the load-bearing test removes the marker entirely and still demands correct state.

Test	What it asserts
test*replay*after*lost*marker*leaves*state_unchanged	**The load-bearing one.** The marker write raises after the MERGE has landed; the identical offset range is replayed with no marker to short-circuit on, and observable state must be unchanged. An extra Iceberg commit is accepted and asserted as such; a changed row is not.
test*replay*with*marker*writes*no*new_snapshot	The happy path: a marked batch is skipped outright, writing no second data commit and no duplicate marker.
test*stale*batch*cannot*revert	An older source LSN can never overwrite a row already updated by a newer one, regardless of arrival order.
test*delete*is*ordered*like*any*other_version	A tombstone obeys the same ordering guard as an update, so a late stale insert cannot resurrect a deleted key, and the key stays single-rowed.
test*in*batch*collapse*keeps*the*newest_version	Several changes to one key inside one batch collapse to the newest, not the oldest — the difference between a correct table and one that silently lags by a batch.
test*toast*placeholder*does*not*overwrite*a*real*value	**Across batches:** a later batch that changes only one column, leaving another TOASTed, must not write Debezium's sentinel string over a real value.
test*toast*placeholder*within*one*batch*carries*the*real_value	**Within one batch:** the max-*lsn row is not the newest* value_ per column. Three events where only the oldest carries a real status must still produce that status.
test*genuine*null*is*still_applied	The other half of the guarantee: a column genuinely set to NULL still writes NULL. A naive placeholder filter would silently refuse every real null-ing.
test*target*is*configured*for*merge*on_read	The target was created merge-on-read, and the startup assertion rejects a copy-on-write table. Correct-but-expensive is the failure mode nobody notices.
test*batch*id*is*pure*and*range*order*insensitive	No Spark required: the id is stable across calls, insensitive

	to the order partition ranges are listed in, and distinct for a different range or topic. If this drifts, every short-circuit above is void.
test*empty*batch*is*skipped	Every empty range hashes identically, so one marker would suppress all later empty batches. Returning early stops that from mattering.

1.1 How each fails if the design were wrong

Break the design this way	**What fails**
Drop the `s.*lsn* > *t.*lsn` predicate	test*stale*batch*cannot*revert **and** test*replay*after*lost*marker*leaves*state_unchanged. This is the scenario that corrupts a real balance, and the one nothing else in the system would notice.
Change `>` to `>=`	A same-version reapply now rewrites the row, so the replay-after-lost-marker test fails on data*snapshot*count. *It does not fail on state* — which is exactly why the snapshot assertion is there.
Make `batch_id()` impure	Add a wall-clock or attempt counter and the replay mints a fresh id, the probe misses, and test*replay*with*marker*writes*no*new_snapshot fails.
Mark the batch applied **before** the MERGE	A crash in between would skip a batch that never landed. **This loses data rather than duplicating work, and I want to be straight that it is not covered:** the ordering is not injectable, so it is asserted by reading apply*micro*batch. test*foreign*marker*does*not*suppress*a_batch is the weaker thing I could actually test.
Physically delete on a Debezium 'd'	The stale insert no longer matches a target row and is inserted as live, so test*delete*is*ordered*like*any*other_version fails on both the flag and the row count.
Revert the collapse to row-wise `ROW_NUMBER()`	test*toast*placeholder*within*one*batch*carries*the*real*value fails* — *the max*-lsn row carries the placeholder because that event never sent a real value either.
Restore `UPDATE SET *` in the MERGE	test*toast*placeholder*does*not*overwrite*a*real*value fails. The in-batch fix alone is not enough; a placeholder arriving as the first event for a key in a batch still needs the changed-flag guard.
Treat the placeholder as null and skip nulls	test*genuine*null*is*still_applied fails — a quieter bug than the one it fixes, because refusing a real null looks like nothing happening.

2. The publish gate — tests/test_stalled_upstream_holds_publish.py

Test	**What it asserts**
test*stalled*upstream*is*held	Healthy loan feed, repayment feed three days behind: HELD, and the candidate branch is cleaned up rather than

	left behind.
test*hole*in*upstream*history*is*held	**The subtle one, and the case a suite written against the obvious bug would miss entirely.** payments*core has reported every date up to D except D-2. max(business*date) says D and publishes a close built on a missing day; the watermark has to say D-3.
test*unknown*upstream*is*not*treated*as_current	An upstream that has never reported yields an unknown watermark, not "whatever date happens to be latest", and still resolves to HELD.
test*p0*failure*holds*and*main*is_untouched	Every upstream complete, so the gate actually reaches the check suite, and the data carries a negative paise value. HELD, main's row count unchanged, valid_through null, and findings asserted to be valid JSON — str(dict) emits single quotes and would corrupt the manifest row it was supposed to explain.
test*publishes*once*the*laggard*catches*up	Once the laggard catches up the same partition publishes, valid_through is set, and the branch is cleaned up. **The gate must not be a one-way ratchet** that stays stuck after the underlying problem is fixed.
test*override*publishes*incomplete*data*but*says_so	The escape hatch works and is on the record: status OVERRIDDEN rather than PUBLISHED, with the approver and ticket in the reason.
test*override*cannot*waive*a_p0	The same override against data with a negative principal is refused. Incompleteness is a judgement call, a broken money column is not, and **that line is policy, so it gets a test rather than a comment.**
test*valid*through*drops*back*when*a*published*date*is*later_held	A date that published on Monday and is held on Tuesday stops being served, because the serving view reads the latest decision per date rather than the existence of a publish.

2.1 How each fails if the design were wrong

Break the design this way	**What fails**
`effective_watermark` takes max instead of min	test*stalled*upstream*is*held fails at the HELD assertion: the stalled feed is ignored and the mart publishes on the healthy feed's schedule — precisely the plausible-but-wrong output this problem exists to prevent.
`source*watermark*` *reverts to* `max(business*date)`	test*hole*in*upstream*history*is*held fails. Same class of failure one level down, and **much easier to ship without noticing, because the newest date really is complete.**
Missing watermark coerced to today	test*unknown*upstream*is*not*treated*as_current fails: silence from an upstream becomes indistinguishable from that upstream being fully current.
Run checks against main, or fast_forward before checking	test*p0*failure*holds*and*main*is_untouched fails, and it fails exactly when a consumer would otherwise see a partition the gate meant to refuse.
Hardcode the catalog in the fast_forward call	test*publishes*once*the*laggard*catches*up and both override tests fail on the catalog name. Worth stating because it is the natural thing to type when the procedure call is

	written out, and every gate test would then error on setup rather than on a claim.
Allow an override to waive a P0	test*override*cannot*waive*a_p0 fails.

The fixtures monkeypatch paging and warning to no-ops, so assertions are on the manifest and on main's row count — the consumer-visible contract — rather than on whether a page fired, which is an implementation detail of the alerting integration. The mart is seeded with an earlier partition so “main is untouched” runs against a non-empty baseline: against zero rows, a bug that writes nothing would also pass.

3. Restatement and knowledge time — tests/test_restatement_classification.py

Test	**What it asserts**
test*renamed*but*identical*file*is*a_duplicate	Same bytes under a new name is a no-op. Partners rename and re-zip constantly, and **only the bytes are evidence.**
test*redelivery*while*first*load*is*in*flight*is*a*duplicate	**The failure mode behind the CLAIMED-as-well-as-COMPLETE rule.** The first claim is CLAIMED, not yet COMPLETE; a lookup restricted to COMPLETE returns nothing, the redelivery reads as brand new, and the same file loads twice concurrently into the same partition.
test*new*content*for*a*known*date*is*a_restatement	Same date, different bytes is a correction and never an append — the default behaviour of every naive file loader, and the one that silently doubles a day's settlement totals.
test*restatement*takes*the*next*knowledge*version	Versions count forward across successive corrections rather than resetting or colliding.
test*second*worker*does*not*own*the_claim	Two workers see the same drop and exactly one owns it. The loser's MERGE is a legitimate no-op and therefore does not raise, so **“did not raise” is worthless as proof of ownership**; the test asserts on the worker_id read back and on there being exactly one ledger row.
test*tag*survives*ordinary*snapshot_expiry	The audit story end to end: cut a tag, write more, expire every ordinary snapshot, read the tag. If RETAIN were missing or in the wrong unit the expiry would drop the snapshot the tag points at, and the read would fail instead of returning what we believed in March.

3.1 How each fails if the design were wrong

Break the design this way	**What fails**
Key the ledger on filename instead of content hash	test*renamed*but*identical*file*is*a_duplicate fails, and in production the day is counted twice.
Filter the ledger lookup to `status = 'COMPLETE'`	test*redelivery*while*first*load*is*in*flight*is*a*duplicate fails.
Treat a new hash for a known date as an append	test*new*content*for*a*known*date*is*a_restatement fails.
Trust “my MERGE did not raise” as proof of ownership	test*second*worker*does*not*own*the_claim fails, and two

	workers load the same file concurrently.
Drop `RETAIN` from the tag DDL	test*tag*survives*ordinary*snapshot_expiry fails — the audit guarantee quietly expires after seven days.

4. Coverage, and what is not tested

All three files test behaviour **through the public entry points** — apply*micro*batch, publish*partition, classify and claim — rather than internal helpers, because the guarantees being defended are properties of that boundary. A helper could be refactored entirely and these would still be the right tests to have. The one deliberate exception is test*batch*id*is*pure*and*range*order_insensitive, which pins an internal detail: purity is not observable from outside, and every replay short-circuit depends on it.

Not tested	**Why, and what it would take**
The suite has not been executed	Written against a real in-process catalog rather than mocks, precisely so it would catch a wrong API rather than just a wrong idea — but no Spark was available in the environment it was written in. I would expect a handful of first-run failures on exact branch DDL and procedure argument shapes. **Closing this is the first thing I would do.**
The three composition points	run*stream, ingest*delivery and publish*restatement wire the guarantees together — publish*restatement in particular gates a correction and only then cuts the tag and emits invalidations, which is what makes the knowledge-time guarantee true rather than aspirational. The pieces are each covered; the wiring is not.
Overwrite atomicity under concurrent readers	That a reader querying mid-overwrite never sees the partition empty or doubled depends on Iceberg's commit semantics behaving exactly as documented under concurrent snapshot creation. Provoking it deterministically needs a concurrent reader thread racing a writer — a test for a real cluster, not local[2], where the scheduler makes the interleaving I care about unlikely.
Marker-before-MERGE ordering	The failure that loses data rather than duplicating work. The ordering is not injectable, so it is asserted by reading the code.
Alerting and override approval	Monkeypatched to no-ops in the fixtures. Asserting a page fired would test the alerting integration, not the gate's contract.